

TP Jour 1 — Introduction au NoSQL & MongoDB (4 h)

Module MIA4 · Conception et intégration d'un SGBD NoSQL — IPSSI Montpellier Jour 1 / 5 — CRUD, Shell Mongo, Docker, JSON/BSON

Contexte professionnel

Vous venez d'être recruté·e comme **Data Engineer junior** par la **Direction de la Santé publique de la Ville de New York (NYC DOHMH)**. Le service dispose d'un export brut, **réel et non nettoyé**, de **25 359 restaurants** et de leurs **inspections d'hygiène**. Ce jeu de données est le *primer dataset* officiel de MongoDB (source : github.com/mongodb/docs-assets).

Votre première mission : **charger la donnée dans MongoDB, l'explorer, répondre à des questions métier précises, corriger la qualité des données et livrer un rapport reproductible**. Comme toute donnée réelle, elle contient des **incohérences volontaires** (arrondissements manquants, notes négatives, lettres de grade aberrantes...). Vous devrez les *détecter par la requête*, pas les deviner.

Attention — toutes les réponses chiffrées de ce TP sont vérifiables : elles correspondent à un état précis de la base. Un chiffre approximatif ne sera pas accepté : vous devez rendre la **commande exacte** ET le **résultat exact**.

Objectifs pédagogiques (rappel du cours)

- Déployer MongoDB via **Docker** (approche reproductible) et importer un jeu réel.
- Se connecter en **mongosh** et en interface graphique (**mongo-express / Compass**).
- Maîtriser les **4 piliers CRUD** et les opérateurs (`$gt`, `$in`, `$exists`, `$size`, `$elemMatch`, `$set`, `$inc`, `$push`, `$pull`...).
- Interroger des **sous-documents** et des **tableaux** (dot-notation, index positionnel).
- Écrire un **script** `.js` exécutable par `mongosh`.
- Relier la pratique aux concepts : **5 V**, **théorème CAP/BASE**, **familles NoSQL**, **JSON vs BSON**.

Pré-requis techniques

- Docker Desktop installé et **démarré**.
- `mongosh` (fourni dans l'image Docker — vous l'utiliserez via `docker exec`, ou installé en local).
- Un éditeur de texte / VS Code. Connexion Internet pour télécharger le jeu de données.

Livrables (à rendre sur Teams avant 17h00)

1. `reponses_jour1.md` — pour **chaque question**, la **commande mongosh** et le **résultat exact**.
2. `rapport.js` — script mongosh de la Partie 5 (exécutable via `mongosh < rapport.js`).
3. `capture_express.png` — capture de mongo-express (ou Compass) montrant la collection `restaurants`.
4. Vos réponses aux **3 questions de réflexion** (Partie 6).

Partie 0 — Mise en place de l'environnement (≈ 30 min)

0.1 Lancer l'infrastructure

Un fichier `docker-compose.yml` est fourni à côté de ce TP (MongoDB 7.0 + mongo-express). Dans le dossier :

```
docker compose up -d
docker compose ps      # les 2 conteneurs doivent être "running"
```

0.2 Récupérer le jeu de données réel

```
curl -L -o primer-dataset.json \
  https://raw.githubusercontent.com/mongodb/docs-assets/primer-dataset/primer-dataset.json
wc -l primer-dataset.json  # vous devez obtenir 25359 lignes
```

0.3 Importer dans MongoDB

Copiez le fichier dans le conteneur puis importez-le dans la base `nyc`, collection `restaurants` :

```
docker cp primer-dataset.json mongo-ipssi:/tmp/primer-dataset.json
docker exec mongo-ipssi mongoimport \
  --username admin --password ipssi2025 --authenticationDatabase admin \
  --db nyc --collection restaurants --drop --file /tmp/primer-dataset.json
```

0.4 Se connecter — les deux clients

- **Shell** : `docker exec -it mongo-ipssi mongosh -u admin -p ipssi2025 --authenticationDatabase admin`
- **Graphique** : ouvrir `http://localhost:8081` (mongo-express). (Alternative : MongoDB Compass avec l'URI `mongodb://admin:ipssi2025@localhost:27017/?authSource=admin`.)

Dans mongosh :

```
use nyc
db.restaurants.countDocuments({})
```

Point de contrôle P0 — Vous devez voir **25359**. Notez-le et faites la capture d'écran de mongo-express (livrable 3). Regardez un document avec `db.restaurants.findOne()` : repérez les champs `address` (sous-document avec `coord`), `borough`, `cuisine`, `grades` (tableau de sous-documents), `name`, `restaurant_id`.

Partie 1 — Lecture & opérateurs (≈ 55 min)

Répondez avec la commande **et** le nombre exact. Utilisez `countDocuments()` (pas l'estimation) sauf indication contraire.

- **Q1.** Combien de restaurants au total ?
- **Q2.** Combien de **types de cuisine distincts** ? (indice : `db.restaurants.distinct("cuisine").length`)
- **Q3.** Combien de restaurants dans l'arrondissement **Brooklyn** ?
- **Q4.** Combien de restaurants de cuisine **French** (exactement) ?
- **Q5.** Combien de restaurants à la fois à **Manhattan** ET de cuisine **Italian** ?
- **Q6.** Combien de restaurants dans **Bronx** ET cuisine **Chinese** ?
- **Q7.** Combien de restaurants ont exactement le nom "**Subway**" ? Puis affichez les **3 premiers** en ne renvoyant que `name` et `borough` (masquez `_id`).
- **Q8.** Avec l'opérateur `$in`, combien de restaurants de cuisine parmi `Japanese`, `Korean`, `Thai`, `Indian` ?
- **Q9. Première question d'écart — le champ de recherche qui ment.** Un usager tape `BBQ` dans la barre de recherche de votre application. Vous branchez naïvement un `$regex` :
 - (a) `db.restaurants.countDocuments({ name: /BBQ/ })` — combien de résultats ? **Le chiffre va vous surprendre.**

- (b) Recommencez en **insensible** à la casse (`/BBQ/i`). Combien maintenant ?
- (c) **Donnez l'écart** et affichez 3 des restaurants que seule la version (b) trouve. Comment sont-ils réellement orthographiés dans la base ?
- (d) Refaites la mesure avec le terme `House` (`/House/` puis `/House/i`) : l'écart y a une **cause différente**. Laquelle ? (Regardez les noms que seule la version *insensible* ramène.)
- (e) Vous devez livrer la recherche. Prenez-vous la version (a) ou (b) ? Sachant qu'un `$regex` **non ancré** ne peut de toute façon pas faire de recherche par préfixe dans un index, et que le drapeau `i` interdit en plus d'exploiter un index standard, quelle **troisième solution** proposeriez-vous en production ? (Vous la construirez au Jour 2.)
- **Q10.** Combien de restaurants dans le code postal (`address.zipcode`) `"10462"` ? (dot-notation sur sous-document)
- **Q11.** Quel est le `name` du restaurant dont le `restaurant_id` vaut `"30075445"` ?

Rappel cours — projection : `db.coll.find( <filtre>, { name:1, borough:1, _id:0 } )`. Tri/limite : `.sort({ name:1 }).limit(5)`.

Partie 2 — Tableaux & sous-documents (≈ 55 min)

Le champ `grades` est un **tableau de sous-documents** `{ date, grade, score }`. C'est ici que se cachent les vrais pièges du modèle document.

- **Q12.** Combien de restaurants ont **au moins une** note dont le `score` est **strictement supérieur à 50** ? (dot-notation `"grades.score"`)
- **Q13. « Mal noté » — mais quand ?** Un journaliste vous demande combien de restaurants sont **mal notés**. Deux lectures possibles, deux requêtes :
 - (a) combien ont **au moins un** `grade` égal à `"C"` (à n'importe quel moment de leur histoire) ?
 - (b) combien ont leur **première entrée** du tableau (`grades.0.grade`) égale à `"C"` ?
 - (c) **Donnez l'écart entre les deux**. Puis ouvrez le tableau `grades` d'un restaurant et **regardez les** `date` : l'entrée d'indice 0 est-elle la plus **ancienne** ou la plus **récente** ? Déduisez-en laquelle des deux requêtes répond à « restaurants **actuellement** mal notés », et laquelle vous publieriez.
- **Q14.** Combien de restaurants ont un tableau `grades` **vide** (`$size: 0`) ? Selon vous, pourquoi une inspection peut-elle produire un tableau vide ?
- **Q15.** Combien de restaurants ont **au moins 6 notes** ? (indice : tester l'existence de l'index positionnel `grades.5` avec `$exists`)
- **Q16.** Combien de restaurants dont la **première** note (`grades.0.grade`) vaut `"A"` ?
- **Q17. Le piège \$elemMatch**. On cherche les restaurants ayant **une même note** qui soit à la fois de `grade` `"B"` ET de `score` `> 20`.
 - (a) Écrivez d'abord la requête *naïve* : `{ "grades.grade": "B", "grades.score": { $gt: 20 } }`. Notez le nombre.
 - (b) Écrivez ensuite la requête correcte avec `$elemMatch`. Notez le nombre.
 - (c) Les deux nombres **diffèrent**. Expliquez en une phrase **pourquoi**, et **lequel** répond réellement à la question métier.
- **Q18. Anomalies de qualité, et ce qu'elles coûtent.** Combien de restaurants ont **au moins une note avec un** `score` **négatif** (`$lt: 0`) ? Un score négatif a-t-il un sens métier ?
 - (b) Mesurez leur impact : calculez le **score moyen** de tout le jeu, une fois **avec** les notes négatives et une fois **sans**. (Indice : `db.restaurants.aggregate([ { $unwind: "$grades" }, { $group: { _id: null, moy: { $avg: "$grades.score" } } } ])`, puis le même pipeline avec un `$match` intercalé.) **Donnez les deux moyennes et l'écart en pourcentage**.
 - (c) Vu cet écart, ces anomalies justifient-elles un nettoyage en urgence ? Argumentez avec **votre chiffre**, pas avec une intuition.
- **Q19.** En utilisant `find(...).sort(...).limit(1)` avec une projection, trouvez le **restaurant dont la note maximale** (`grades.score`) **est la plus élevée de tout le jeu**. Donnez son `name` et ce score. (Indice : le tri décroissant sur un champ tableau utilise la valeur maximale du tableau.)

Partie 3 — Création & mise à jour (≈ 40 min)

Lisez ceci avant de commencer. À partir d'ici vous **modifiez la base** : vous allez insérer un restaurant, ajouter des notes, poser des champs et supprimer des documents. **Vos comptages ne correspondront donc plus à ceux de la Partie 1**, et c'est normal. Notez au fur et à mesure ce que chaque opération change ; la Partie 5 vous demandera d'expliquer l'écart entre le total de la Q1 et celui de votre rapport final.

- **Q20. CREATE.** Insérez **votre propre restaurant fictif** (avec `insertOne`) : un `name` contenant vos initiales, `borough: "Montpellier"`, `cuisine: "French"`, un `address` avec `coord: [3.8767, 43.6108]` (Montpellier), et un tableau `grades` avec **une** note `{ grade: "A", score: 7, date: new Date() }`. Vérifiez avec `findOne` sur votre `name`.
- **Q21. UPDATE ciblé.** Sur le restaurant `restaurant_id: "30075445"`, **ajoutez** une nouvelle note en fin de tableau avec `$push` : `{ grade: "A", score: 3, date: new Date() }`. Combien de notes ce restaurant a-t-il maintenant ? (*il en avait 5*)
- **Q22. UPDATE de masse.** Ajoutez le champ `risque: "eleve"` (`$set`) à **tous** les restaurants ayant au moins une note de `score > 50`. Donnez `matchedCount` et `modifiedCount` renvoyés par `updateMany`.
- **Q23. UPDATE conditionnel.** Sur tous les restaurants de cuisine **French**, ajoutez le champ `label_qualite: true`. Combien ont été modifiés ?

Rappel cours : `updateOne(<filtre>, { $set:{...}, $inc:{...}, $push:{...} })` — n'oubliez pas les **opérateurs** ; un `update` sans opérateur **remplace tout le document**.

Partie 4 — Suppression & qualité de données (≈ 20 min)

Les 51 documents avec `borough: "Missing"` sont des enregistrements corrompus lors de l'export : impossible de les rattacher à un arrondissement.

- **Q24.** Comptez d'abord les documents `borough: "Missing"`. Combien ?
- **Q25.** Supprimez-les avec `deleteMany`. Donnez `deletedCount`, puis **re-comptez** la collection entière. Combien de documents reste-t-il ?
- **Q26. Décision de gouvernance.** Vous avez compté les tableaux `grades` vides en Q14. **Ne les supprimez pas** : décidez.
 - **(a)** Rapportez leur nombre à l'effectif **actuel** de la collection (après la Q25) : quel **pourcentage** représentent-ils ?
 - **(b)** Ces documents et les 51 `borough: "Missing"` de la Q24 sont deux anomalies très différentes. Vous en avez supprimé une et vous conservez l'autre : **justifiez ce traitement asymétrique en une phrase**. (*Qu'est-ce qui est irrécupérable dans un cas et pas dans l'autre ?*)

Partie 5 — Automatisation : script `.js` + export (≈ 40 min)

- **Q27.** Écrivez un fichier `rapport.js` (mongosh) qui, à l'exécution, **affiche un rapport** contenant, via des `print()` / `console.log()` et des variables JS :
 1. le nombre total de restaurants,
 2. le **top 5 des cuisines** les plus fréquentes (utilisez une boucle JS sur `db.restaurants.distinct("cuisine")` + `countDocuments`, ou un `Map`),
 3. le nombre de restaurants par arrondissement (bouclez sur `db.restaurants.distinct("borough")`).

Puis, dans `reponses_jour1.md` : votre total (1) **diffère** de celui de la Q1, et votre liste d'arrondissements (3) contient une valeur qui n'existait pas au départ. **Donnez l'écart exact sur le total et expliquez-le opération par opération** (quelle question a ajouté ou retiré combien de documents ?). Un rapport dont on ne sait pas reconstituer les chiffres n'est pas un rapport.

Il doit s'exécuter ainsi :

```
docker exec -i mongo-ipssi mongosh -u admin -p ipssi2025 --authenticationDatabase admin nyc < rapport.js
```

- **Q28.** Exportez la collection `restaurants` limitée à l'arrondissement **Staten Island** vers un fichier JSON avec `mongoexport` (livrable optionnel : joignez le fichier). Combien de lignes contient l'export ?

Partie 6 — Réflexion : relier la pratique au cours (≈ 20 min)

Rédigez **4–6 lignes par question** (dans `reponses_jour1.md`).

Règle de correction, annoncée d'avance : chacune de ces trois questions doit citer **au moins un chiffre que vous avez vous-même relevé**, avec le numéro de la question dont il vient. Une réponse générale et bien tournée, mais sans aucun de vos résultats, **ne vaudra pas de points** — c'est une dissertation, pas une analyse.

- **R1. Les 5 V, chiffrés.** Illustrez **au moins 3 des 5 V** (Volume, Vitesse, Variété, Vérité, Valeur) en vous appuyant **chacun sur un résultat numérique de ce TP** et en citant sa question. Par exemple, la **Vérité** ne s'illustre pas par « il y a des erreurs » mais par « X documents portent un score négatif (Q18a), ce qui déplace le score moyen de Y % (Q18b) ». Faites de même pour les deux autres V que vous choisirez.
- **R2. CAP & BASE, appliqué à ce service.** MongoDB est **CP** par défaut. Pour l'application publique qui consulte ces inspections d'hygiène, préféreriez-vous privilégier la **cohérence (C)** ou la **disponibilité (A)** en cas de partition réseau ? Ne répondez pas dans l'abstrait : **construisez le scénario**. Prenez le restaurant de la Q11 et imaginez qu'il vienne d'être fermé pour insalubrité. Décrivez concrètement ce que voit l'utilisateur (a) si l'on a choisi C, (b) si l'on a choisi A. Puis tranchez, et dites **quel type de dommage** vous avez accepté en tranchant.
- **R3. Embarqué vs référencé — le calcul.** Les `grades` sont **embarqués** dans chaque restaurant.
 - **(a)** Reprenez votre réponse à la Q15 (restaurants ayant au moins 6 notes) et celle de la Q21 (nombre de notes après votre `$push`). Estimez la **taille en octets** d'une note (regardez un document avec `Object.bsonsize(db.restaurants.findOne())` et divisez).
 - **(b)** Un restaurant inspecté **chaque semaine pendant 10 ans** accumulerait 520 notes. Quelle taille atteindrait son document ? Comparez à la **taille maximale d'un document BSON** — donnez cette limite. Le modèle embarqué tient-il ?
 - **(c)** À partir de ce calcul, donnez **un avantage** et **une limite** du choix embarqué, et dites **à partir de quel volume** vous basculeriez vers un modèle référencé.

Pour aller plus loin (facultatif)

- **B1. Index.** Créez un index sur `cuisine` (`createIndex`). Comparez le plan d'exécution d'une recherche `cuisine:"French"` avant/après avec `.explain("executionStats")` : quel `stage` passe de `COLLSCAN` à `IXSCAN` ? Que devient `totalDocsExamined` ?
- **B2. Géospatial.** Créez un index `2dsphere` sur `address.coord` et trouvez les restaurants situés à moins de **500 m** d'un point donné (`$near`).

Checklist finale avant rendu

`reponses_jour1.md` : Q1→Q28 avec **commande + résultat exact** + R1→R3 **citant vos chiffres**

Les écarts sont expliqués : Q9(c), Q13(c), Q18(b), Q27 (total final vs Q1)

`rapport.js` fonctionnel

`capture_express.png`

Environnement Docker reproductible (le `docker-compose.yml` et les commandes d'import figurent dans le rendu)